

wa-final (grupa A)

FIPU Web aplikacije: *final kolokvij* (28. 1. 2025.)

Ime i prezime: _____

JMBAG: _____

Potpis: _____

VAŽNA NAPOMENA: Predajete samo 1 zip datoteku ukupnog projekta. Prije predaje i zippanja, **obavezno obrišite direktorij** `node_modules`.

Predajete samo jednu zip datoteku naziva: `wa-final-grupaA-ime_prezime.zip` na Google Forms poveznici.

Zadatak 1 (10 bodova)

Klonirajte Github repozitorij `lukablaskovic/wa-final-template` (<https://github.com/lukablaskovic/wa-final-template.git>) i pokrenite ga lokalno u vašem razvojnom okruženju.

Dodajte datoteku `.env` u kojoj ćete prema predlošku iz `.env-template` postaviti vrijednost varijable okruženja za MongoDB Connection string: `MONGO_URI`.

Izradite **bazu podataka** `wa_final` s kolekcijom `users` na Atlasu.

Nakon instalacije svih paketa, pokrenite poslužitelj naredbom `npm run dev` ili `node index.js`.

Ako je sve u redu, trebali biste dobiti poruku: `Uspješno spajanje na bazu podataka` u terminalu.

Unutar `routes/users.js` definirajte sljedeću rutu:

1.1 `POST /fake` koja dodaje novog fake korisnika u Mongo kolekciju `users`.

- Nakon uspješnog dodavanja vratite klijentu odgovarajući statusni kod i novododanog korisnika u tijelu odgovora.

Svaki korisnik u kolekciji treba imati sljedeće atribute: `id` (Mongo ID), `username`, `email`, `avatar`, `birthdate`, `registeredAt`

Podatke o korisniku ne šalje klijent u HTTP tijelu zahtjeva, već se generira fake korisnik na poslužiteljskoj strani pomoću `faker` biblioteke.

```
npm install @faker-js/faker
```

Za generiranje fake korisnika pozovite sljedeću funkciju:

```
import { faker } from '@faker-js/faker';
function createRandomUser() {
  return {
    username: faker.internet.username(),
    email: faker.internet.email(),
    avatar: faker.image.avatar(),
    birthdate: faker.date.birthdate(),
    registeredAt: faker.date.past(),
  };
}
```

Uključite router u glavnu aplikaciju s odgovarajućim prefiksom.

Primjer HTTP **odgovora**:

```
{
  "username": "Rosanna74",
  "email": "Shad_OConnell150@gmail.com",
  "avatar": "https://avatars.githubusercontent.com/u/26130322",
  "birthdate": "1980-07-24T04:20:36.347Z",
  "registeredAt": "2024-04-23T16:41:02.250Z",
  "_id": "6797572feffdc8ccf4d0def7"
}
```

Zadatak 2 (15 bodova)

Dodajte 10 fake korisnika u kolekciju `users` i definirajte sljedeće rute unutar `routes/users.js`:

2.1 `GET /` koja vraća sve korisnike iz kolekcije `users` u obliku polja objekata.

- Dodajte **query** parametar `limit` koji ograničava broj korisnika koji se vraćaju u odgovoru. Primjerice, ako klijent poslužitelju pošalje zahtjev `GET /?limit=5`, poslužitelj vraća prvih 5 korisnika iz kolekcije.
- Pretraživanje u bazi podataka možete ograničiti na određeni broj dokumenata pomoću metode `limit()`.

```
collection.find().limit(5);
```

- Ako query parametar `limit` nije definiran, vratite sve korisnike iz kolekcije.

2.2 `GET /:id` koja vraća korisnika iz kolekcije `users` prema ID-u.

- `id` se odnosi na `Mongo ID` korisnika u kolekciji `users`.
- Koristeći biblioteku `express-validator` validirajte **je li ID proslijeđen, je li string i ima li točno 24 znaka**.
- u slučaju uspješnog dohvaćanja vratite odgovarajući statusni kod i podatke o korisniku. Ako korisnik s traženim ID-om ne postoji, vratite odgovarajući statusni kod i poruku o grešci.

2.3 `DELETE /:id` koja briše korisnika iz kolekcije `users` prema ID-u.

- u slučaju uspješnog brisanja vratite odgovarajući statusni kod i poruku o uspješnom brisanju. Ako korisnik s traženim ID-om ne postoji, vratite odgovarajući statusni kod i poruku o grešci.

Zadatak 3 (10 bodova)

3.1 U datoteci `middleware/middleware.js` definirajte **middleware funkciju** koja će pronalaziti korisnika prema ID-u i dodavati ga u objekt `req` za sve rute koje sadrže parametar `:id`. Osim logike za pronalazak korisnika, u *middleware* prebacite logiku validacije ID-a.

- *middleware* možete nazvati `findUserById` i koristiti ga u svim rutama koje sadrže parametar `:id`.

Primjer korištenja:

```
router.get('/:id', [findUserById], (req, res) => {  
  const user = req.user;  
});
```

3.2 Dodajte *middleware* `logger` na razini glavne Express aplikacije koji će ispisivati u konzolu informacije o svakom zahtjevu koji dolazi na poslužitelj.

middleware mora ispisivati informacije u sljedećem formatu:

- trenutni datum i vrijeme
- HTTP metoda (`req.method`)
- URL zahtjeva (`req.originalUrl`)
- statusni kod odgovora (`res.statusCode`)

Primjer:

```
[2025-01-27T10:49:29.811Z] POST /drinks/fake 404
```

Zadatak 4 (10 bodova)

Definirajte datoteku `auth.js` u kojoj ćete implementirati **dvije nove funkcije**:

4.1 `hashPassword` koja prima lozinku kao argument i vraća hashirani oblik lozinke pomoću `bcrypt` biblioteke.

- Nadogradite rutu iz **Zadataka 1.1** na način da klijent šalje lozinku u **tijelu HTTP zahtjeva** i da se lozinka sprema u bazi podataka **u hashiranom obliku** (naravno, uz preostale fake podatke generirane na poslužitelju).
- Validirajte dolaznu lozinku koristeći biblioteku `express-validator` i dodajte pravilo da lozinka mora imati **minimalno 6 znakova** i **sadržavati isključivo alfanumeričke znakove**.

4.2 `comparePasswords` koja prima dva argumenta: `password` i `hashedPassword` te uspoređuje lozinku s hashiranim oblikom lozinke.

Zadatak 5 (15 bodova)

Unutar `routes/users.js` definirajte sljedeću rutu:

5.1 `POST /login` koja će autentificirati korisnika prema **korisničkom imenu i lozinci** ili **prema emailu i lozinci** (ovisno što korisnik unese u tijelu zahtjeva).

Primjer tijela HTTP zahtjeva:

```
{
  "username": "Rosanna74",
  "password": "password123"
}

// ili

{
  "email": "Shad_OConnell150@gmail.com",
  "password": "password123"
}
```

- Provjerite je li korisnik unio korisničko ime ili email i lozinku, prvo pretražite korisnika u bazi podataka po jednom od ta dva ključa, **ako korisnik ne postoji** vratite odgovarajući statusni kod i poruku o grešci.
- **Ako korisnik postoji**, koristite funkciju `comparePasswords` iz **Zadatka 4.2** za usporedbu lozinke iz zahtjeva s hashiranom lozinkom iz baze podataka. Ako je korisnik unio pogrešnu lozinku, vratite odgovarajući statusni kod i poruku o grešci.
- **Ako je korisnik unio ispravne podatke**, generirajte JWT token pomoću `jsonwebtoken` biblioteke i vratite ga klijentu u tijelu odgovora.

5.2 Dodajte *middleware* `authUser` koja će **provjeravati ispravnost JWT tokena** koji je proslijeđen u **zaglavlju (headers) HTTP zahtjeva**. Ako je token neispravan, vratite korisniku odgovarajući statusni kod i poruku o grešci.

- *middleware* ubacite za rute: `GET /`, `GET /:id`, `DELETE /:id` u `routes/users.js`.

Primjer HTTP zahtjeva s autorizacijskim zaglavljem:

```
curl -X GET http://localhost:3000/ -H "Authorization: Bearer <token>"
```

- zahtjeve s JWT tokenom u zaglavlju možete poslati i kroz neki od HTTP klijenata